

Demo: Building ETL Routes using FastAPI

1. Overview

In this demo, learners will implement an **ETL pipeline as REST APIs using FastAPI**. The pipeline will allow users to upload data, apply transformations, and load the processed data into a **SQLite database**.

The demo builds three main APIs:

- **Extract API** – uploads and reads data from a CSV file
- **Transform API** – applies filtering and column transformations
- **Load API** – stores the processed data into a SQLite database

The entire pipeline is executed through **FastAPI endpoints and tested using Swagger UI**.

2. Learning Objectives

By the end of this demo, learners will be able to:

- Implement an **ETL workflow using REST APIs**
- Upload files using **FastAPI multipart file upload**
- Use **Pandas** to process and transform data
- Build APIs for **data extraction, transformation, and loading**
- Store processed data into a **SQLite database**
- Test APIs using **Swagger UI**

3. Project Structure

The FastAPI project contains the following files:

```
project/
|
├─ main.py
├─ requirements.txt
├─ sample.csv
└─ etl.db
```

File descriptions

main.py

Contains the FastAPI application and all ETL API routes.

requirements.txt

Lists all required dependencies.

sample.csv

Sample dataset used for extraction.

etl.db

SQLite database where the processed data will be stored.

4. Dependencies

The project requires the following libraries:

- **fastapi** – framework for building APIs
- **uvicorn** – ASGI server to run FastAPI applications
- **pandas** – data processing and transformation
- **python-multipart** – enables file uploads in FastAPI

Install dependencies using:

```
pip install -r requirements.txt
```

python-multipart is specifically required to **handle file uploads in API requests**.

5. Running the FastAPI Application

Run the application using **Uvicorn**:

```
uvicorn app:app --reload
```

Once the server starts, open the API documentation:

<http://127.0.0.1:8000/docs>

This opens **Swagger UI**, where all APIs can be tested interactively.

6. Extract API – Upload and Read CSV Data

The first step in the ETL pipeline is **data extraction**.

Endpoint

POST /extract-csv

Purpose

- Upload a **CSV file**
- Read the file into a **Pandas DataFrame**
- Generate a **token**
- Store the data in memory using the token

Sample CSV file

```
name,age,city,salary
Vivek,35,Bangalore,60000
Rahul,28,Mumbai,50000
Anita,32,Bangalore,70000
```

Steps in Swagger

1. Open **/extract-csv**
2. Click **Try it out**
3. Upload the CSV file
4. Execute the request

Response

- Returns **dataset preview**
- Returns a **token**

Example response:

```
{
  "token": "abc123-token",
  "data": [...]
}
```

The **token is important** because it will be used to call the next APIs in the pipeline.

7. Transform API – Apply Data Transformations

After extraction, the next step is **data transformation**.

Endpoint

POST /transform

Purpose

Applies transformations such as:

- **Selecting specific columns**
- **Renaming columns**
- **Filtering rows**

Example transformation

Input token from previous step:

token: abc123-token

Transformation rules:

- Select columns **name, age, city**
- Rename **age** → **years**
- Filter rows where:
 - years > 30
 - city contains "ban"

Example response

```
[
  {
    "name": "Vivek",
    "years": 35,
    "city": "Bangalore"
  }
]
```

This shows the filtered and transformed dataset.

The transformed data remains **linked to the same token**, enabling chained transformations.

8. Load API – Store Data in SQLite

The final step in the ETL pipeline is **loading the processed data into a database**.

Endpoint

POST /load

Purpose

- Retrieve transformed data using the **token**
- Write the data into a **SQLite database**

Example request:

token: abc123-token

Response

```
{  
  "status": "Data successfully loaded into SQLite"  
}
```

9. Verifying Data in SQLite

After loading the data, you can verify it in the database.

Open SQLite and run:

```
SELECT * FROM etl_table;
```

Example output:

name	years	city
Vivek	35	Bangalore

This confirms that the **ETL pipeline successfully extracted, transformed, and loaded the data.**

10. Understanding the Application Code

Imports used

The FastAPI application imports several libraries:

- **FastAPI** – API routing
- **Pydantic** – request validation
- **Pandas** – data processing
- **SQLite** – database storage
- **Utilities for file uploads and tokens**

Token mechanism

A **UUID token** is generated when data is extracted.

The token is used to:

- Store the **DataFrame in memory**
- Retrieve it during **transform**
- Load it during **load**

This enables the APIs to work as a **stateful ETL pipeline.**

11. Additional Extraction Endpoint

The project also supports **JSON-based extraction**.

Endpoint

POST /extract-json

Instead of uploading a file, this endpoint accepts a **JSON array of records**, converts it into a **Pandas DataFrame**, and returns a **token**.

12. End-to-End ETL Workflow

The complete ETL flow works as follows:

1. Extract

- a. Upload CSV file
- b. Generate token
- c. Store data in memory

2. Transform

- a. Apply column selection
- b. Rename columns
- c. Filter rows

3. Load

- a. Store processed data into SQLite database

This demonstrates how **FastAPI can expose a full ETL pipeline through REST APIs**.

13. FastAPI Implementation of the ETL Pipeline

In this section, we implement the **ETL pipeline using FastAPI**. The application exposes APIs that allow users to:

- Upload data (**Extract**)
- Apply transformations (**Transform**)
- Store processed data in a database (**Load**)

The application uses:

- **FastAPI** for API creation
- **Pandas** for data processing
- **SQLite** for storage
- **Pydantic** for request validation

The pipeline works by generating a **token** when data is extracted. This token is used to track the dataset across transformation and loading steps.

14. Application Code (main.py)

Below is the complete FastAPI implementation used in this demo.

```
from fastapi import FastAPI, UploadFile, File, HTTPException
from pydantic import BaseModel
import pandas as pd
import sqlite3
import uuid
from typing import List, Dict, Optional

app = FastAPI(title="ETL API", version="1.0")

# In-memory storage for datasets
data_store = {}
```

```
DATABASE = "etl.db"
```

```
# Helper function to retrieve dataframe
```

```
def get_dataframe(token: str):  
    if token not in data_store:  
        raise HTTPException(status_code=400, detail="Invalid token")  
    return data_store[token]
```

```
# -----
```

```
# Extract CSV API
```

```
# -----
```

```
@app.post("/extract-csv")
```

```
async def extract_csv(file: UploadFile = File(...)):
```

```
    contents = await file.read()
```

```
    df = pd.read_csv(pd.io.common.BytesIO(contents))
```

```
    token = str(uuid.uuid4())
```

```
    data_store[token] = df
```

```
    return {
```

```
        "token": token,
```

```
        "preview": df.head().to_dict(orient="records")
```

```
    }
```

```
# -----
```

```
# Extract JSON API
```

```
# -----
```

```
class JSONInput(BaseModel):
```

```
    records: List[Dict]
```

```
@app.post("/extract-json")
```

```

def extract_json(payload: JSONInput):

    df = pd.DataFrame(payload.records)

    token = str(uuid.uuid4())

    data_store[token] = df

    return {
        "token": token,
        "preview": df.head().to_dict(orient="records")
    }

# -----
# Transform API
# -----
class FilterCondition(BaseModel):
    column: str
    operator: str
    value: str

class TransformInstruction(BaseModel):
    token: str
    select: Optional[List[str]] = None
    rename: Optional[Dict[str, str]] = None
    filters: Optional[List[FilterCondition]] = None

@app.post("/transform")
def transform_data(instruction: TransformInstruction):

    df = get_dataframe(instruction.token).copy()

    # Column selection
    if instruction.select:
        df = df[instruction.select]

```

```

# Rename columns
if instruction.rename:
    df = df.rename(columns=instruction.rename)

# Apply filters
if instruction.filters:
    for f in instruction.filters:

        if f.operator == ">":
            df = df[df[f.column] > float(f.value)]

        elif f.operator == "<":
            df = df[df[f.column] < float(f.value)]

        elif f.operator == "contains":
            df = df[df[f.column].str.contains(f.value,
case=False)]

    data_store[instruction.token] = df

    return {
        "message": "Transformation applied",
        "data": df.to_dict(orient="records")
    }

# -----
# Load API
# -----
class LoadRequest(BaseModel):
    token: str
    table_name: str = "etl_table"

@app.post("/load")
def load_to_sqlite(request: LoadRequest):

    df = get_dataframe(request.token)

```

```
conn = sqlite3.connect(DATABASE)

df.to_sql(request.table_name, conn, if_exists="replace",
index=False)

conn.close()

return {
    "status": "Data loaded successfully",
    "table": request.table_name
}
```

15. Installing Dependencies

Create a **requirements.txt** file:

```
fastapi
uvicorn
pandas
python-multipart
pydantic
```

Install dependencies:

```
pip install -r requirements.txt
```

16. Running the Application

Start the FastAPI server using **Uvicorn**:

```
uvicorn main:app --reload
```

Once the server starts, open:

<http://127.0.0.1:8000/docs>

This will open **Swagger UI**, where all ETL APIs can be tested interactively.

17. End-to-End API Workflow

Step 1: Extract Data

Upload a CSV file using:

POST /extract-csv

Response:

```
{  
  "token": "123abc"  
}
```

Step 2: Transform Data

Use the token to apply transformations.

Example request:

```
{  
  "token": "123abc",  
  "select": ["name", "age", "city"],  
  "rename": {"age": "years"},  
  "filters": [  
    {  
      "name": "John",  
      "age": 30,  
      "city": "New York"  
    }  
  ]  
}
```

```
{
  "column": "years", "operator": ">", "value": "30"},
  {"column": "city", "operator": "contains", "value": "ban"}
}
```

Step 3: Load Data

Store the transformed data into **SQLite**.

POST /load

Request:

```
{
  "token": "123abc",
  "table_name": "employees"
}
```

18. Verifying Data in SQLite

After loading the data, open the SQLite database and run:

```
SELECT * FROM employees;
```

Example output:

name	years	city
Vivek	35	Bangalore

This confirms the **ETL pipeline successfully extracted, transformed, and loaded the data.**